

Wykład 2

PTM

Mikrokontroler ATMEL AVR ATMEGA

Na podstawie książki:

Mikrokontrolery AVR ATmega w praktyce - Rafał Baranowski

Zestawienie kryteriów technicznych o największym znaczeniu dla producentów urządzeń wykorzystujących mikrokontrolery



elektronikab2b.pl

Wstęp

AVR – rodzina ośmiobitowych mikrokontrolerów produkowanych przez firmę Atmel – MICROCHIP

Charakterystyka ogólna:

- Rdzeń zaprojektowany z myślą o językach programowania wysokiego poziomu – C
- Architektura RISC (ale w sumie ponad 130 rozkazów ! – brak możliwości prowadzenia operacji arytmetyczno logicznych bezpośrednio na pamięci danych, za to jest szybsza praca i duża liczba rejestrów roboczych)
- Słowo rozkazowe ma długość 16 bitów
- ALU może wykonywać większość instrukcji w trakcie 1 cyklu zegarowego
- ALU posiada układ mnożący (operacja zajmuje 2 takty zegarowe
- ALU posiada bezpośredni dostęp do wszystkich rejestrów roboczych
- Przetwarzanie potokowe
- Prędkość przetwarzania może dochodzić do 1 MIPS (milion operacji na sekundę) na każdy 1 MHz częstotliwości taktującej (a taktowanie może być 20 MHz)

Wikipedia.pl

Wstęp

Układy z grupy AVR zostały podzielone na kilka podgrup:

8 bitowe

tinyAVR – oznaczenie ATtiny

0,5-16 kB pamięci programu

obudowy o 6-32 wyprowadzeniach

ograniczony zestaw zintegrowanych peryferiów

obniżony pobór mocy



ATtiny



ATmega

megaAVR – oznaczenie ATmega

4-256 kB pamięci programu

obudowy o 28-100 wyprowadzeniach

rozszerzony zestaw poleceń (m.in. o polecenia do obsługi powiększonej pamięci programu)

liczne zintegrowane peryferia (przetworniki AC/CA, timery, liczniki, USB itp.)

Wikipedia.pl

Wstęp

8/16 bitowe

AVR XMEGA – oznaczenie ATxmega

8-384 kB pamięci programu

obudowy o 32-100 wyprowadzeniach

rozszerzone możliwości, takie jak obsługa DMA, system zdarzeń (Event System), logika programowalna (XCL) i sprzętowe wsparcie dla kryptografii (używane m.in. przy współpracy z kartami inteligentnymi)

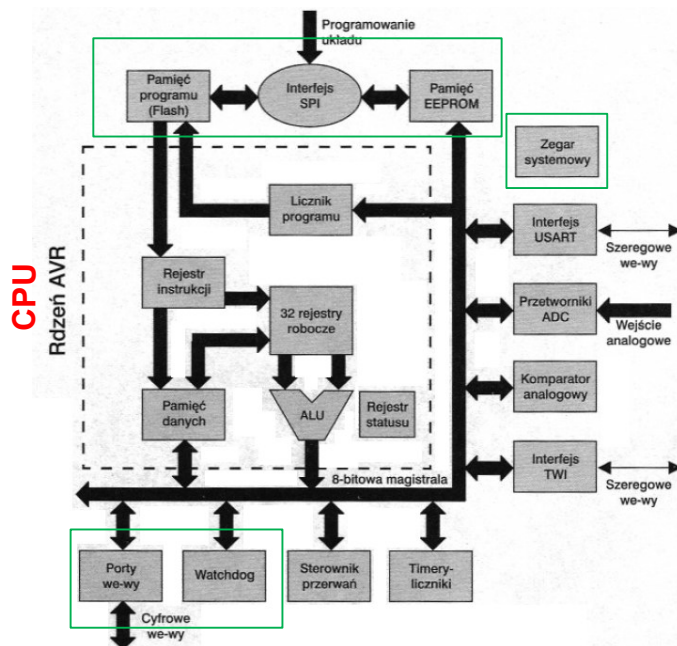


XMEGA12

32 bitowe (lista rozkazów architektury AVR32 jest niekompatybilna z AVR)

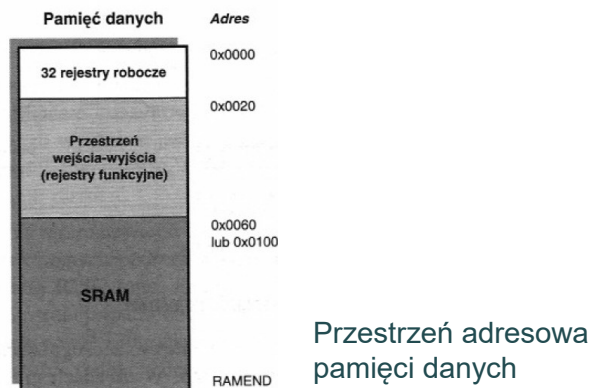
Wikipedia.pl

AVR – schemat blokowy



AVR – rodzaje pamięci

- Architektura harwardzka – rozdzielona pamięć programu i pamięć danych (mniej uniwersalne ale tanie)
- „Hybrydowa” pamięć danych – SRAM połączone z rejestrami roboczymi i funkcyjnymi w jedną przestrzeń adresową



AVR – rodzaje pamięci

Rejestry ogólnego przeznaczenia

- Bezpośredni dostęp ALU do obszaru rejestrów o wielkości 32 bajtów; każdy może pełnić funkcję akumulatora (miejsce pobrania argumentu oraz umieszczenia wyniku większości operacji arytmetyczno logicznych)

Pamięć danych	Adres	Nazwa dodatkowa
R0	0x0000	
R1	0x0001	
R2	0x0002	
...		
R15	0x000F	
R16	0x0010	
...		
R25	0x0019	
R26	0x001A	XL (LSB rejestru X)
R27	0x001B	XH (MSB rejestru X)
R28	0x001C	YL (LSB rejestru Y)
R29	0x001D	YH (MSB rejestru Y)
R30	0x001E	ZL (LSB rejestru Z)
R31	0x001F	ZH (MSB rejestru Z)

- R26-31 2 pary dwubajtowych rejestrów indeksowych używanych jako wskaźniki przy adresowaniu pośrednim
- Dla R0-15 jest ograniczenie, że nie mogą być używane przez ALU przy operacjach ze stałymi ładowanymi bezpośrednio

AVR – rodzaje pamięci

Przestrzeń wejścia-wyjścia

- Znajdują się wszystkie rejestry funkcyjne (specjalne) dostarczające informacji o aktualnym stanie mikrokontrolera, umożliwiające sterowanie pracą rdzenia i wewnętrznych układów peryferyjnych
- Rozmiar duży (typowo 256 B) spowodowany dużą liczbą wewnętrznych układów peryferyjnych, która wymaga znacznej liczby rejestrów funkcyjnych

AVR – rodzaje pamięci

Wewnętrzna pamięć SRAM

- Dostęp do tej pamięci trwa jedynie dwa cykle zegarowe, lecz nie można przeprowadzać na niej operacji arytmetyczno-logicznych w sposób bezpośredni
- Dla ATmega32 wielkość to 2kB

Stos

- Pamięć SRAM stanowi też stos mikrokontrolera (*tymczasowe przechowywanie zmiennych lokalnych oraz zapamiętywanie adresów powrotu przy wywoływaniu podprogramów – automatycznie wykonywane przez mikrokontroler*)

Pamięć programu

- Dla ATmega32 wielkość to 32kB
- Pamięć typu flash w systemie ISP „In-System Programmable” - umożliwiającym zaprogramowanie układu scalonego bez jego demontażu z urządzenia, w którym pracuje.
- Gwarantowana minimalna liczba przeprogramowań – 10 000 razy

AVR – rodzaje pamięci

Bity sterujące

Bity konfiguracyjne – *fuse bits*

- Umożliwiają konfigurację/zmianę wariantu mikrokontrolera

Np.:

Nazwa bitu	Opis	Typowa wartość domyślna
BODEN	Włącza układ zerujący mikrokontroler w przypadku nieodpowiedniego napięcia zasilania (ang. <i>Brown-out Detector</i>). Więcej w rozdziale	1 (BOD wyłączony)
RSTDISBL	Determinuje tryb pracy wyprowadzenia RESET, które w niektórych mikrokontrolerach służyć może również jako port wejścia-wyjścia. Zaprogramowanie tego bitu uniemożliwia późniejsze programowanie szeregowe poprzez interfejs SPI.	1 (RESET działa jako wyprowadzenie zerujące)
SUT1...0	Ustawia opóźnienie z jakim mikrokontroler zaczyna działać po dołączeniu zasilania tak, aby generator taktujący i napięcie zasilania miały czas na ustabilizowanie się (ang. <i>start-up time</i>). Ustawienie powinno być uzależnione od zastosowanego źródła zegarowego – więcej informacji w rozdziale 1.6.7.	10

Bity zabezpieczające – lock bits

- Umożliwiają wprowadzenie układu w tryb ochrony wewnętrznej pamięci programu, aby uniemożliwić osobie niepowołanej odczyt jej zawartości

AVR – budowa

Zegar systemowy

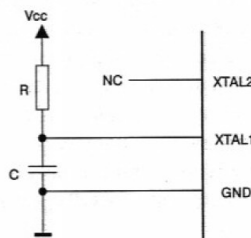
Wewnętrzny, skalibrowany układ RC

- Umożliwia pracę układu bez żadnych elementów zewnętrznych
- Dokładność 3%
- Dodatkowo wpływ zmiany napięcia zasilania i temperatury

Zewnętrzny układ RC

- Lepsza stabilność
- Ręczna możliwość regulacji częstotliwości zegarowej przez potencjometr

$$f = 1/3 \cdot R \cdot C$$



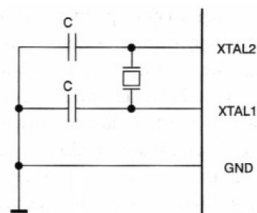
AVR – budowa

Zegar systemowy

Zewnętrzny rezonator kwarcowy i ceramiczny

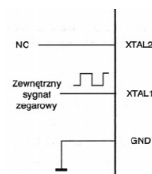
- Największa dokładność i stałość zegara taktującego
- Kwarcowy (dokładność), ceramiczny (odporność – niska temperatura, drgania mechaniczne)

Pojemności obciążające 11-22 pF



Zewnętrzne źródło sygnału zegarowego

- Dla zewnętrznych sygnałach zegarowych o poziomach TTL/CMOS
- Czas narostu i opadania <500 ns
- Zmiana f w czasie jednego cyklu <2%



AVR – budowa

Zegar systemowy

Zewnętrzne wykorzystanie zegara systemowego

- W zależności od wersji **uC** odpowiednia konfiguracja układu zegara; należy stosować bufor lub inwerter w technologii CMOS aby nie obciążać wyjścia

Preskaler częstotliwości zegarowej

- Wewnętrzny układ umożliwiający dzielenie częstotliwości układu zegarowego
- Ograniczenie poboru mocy
- Przydatna w aplikacjach w których wymagania wydajnościowe są zmienne w czasie

Czasy startu układu zegarowego

- Ustawienie opóźnień między włączeniem zasilania układu a startem mikrokontrolera, ze względu na ustabilizowanie się f układu zegarowego

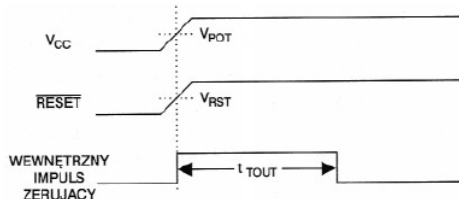
AVR – budowa

Układy zerujące/resetujące

- Funkcja inicjalizacji wszystkich bloków wewnętrznych przed przystąpieniem do pracy: rejestry wejścia-wyjścia ładowane są wartościami domyślnymi, wyprowadzenia portów są w stanie wysokiej impedancji, licznik programu jest zerowany, (*rejestry robocze i pamięć SRAM nie jest zerowana!*)
- po zaniku sygnału zerującego, uC rozpoczyna pracę po odczekaniu czasu opóźnienia t_{TOUT} , w celu ustalenia warunków zasilania i częstotliwości zegara systemowego

Układ POR (power on reset)

- Generuje impuls zerujący tak długo jak napięcie zasilania jest niższe niż napięcie odniesienia (typowo 1,4 V)
- Automatyczna inicjalizacja mikrokontrolera po zaniku zasilania

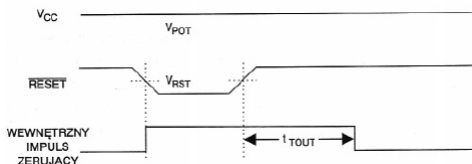


AVR – budowa

Układy zerujące/resetujące

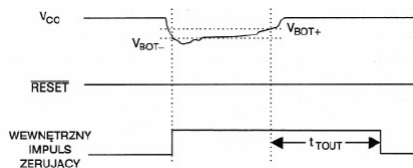
Zewnętrzne źródło zerujące

- Do ręcznego zerowania mikrokontrolera
- Wyprowadzenie RESET aktywne stanem niskim; minimalna długość impulsu zerującego 2,5 ms
- W niektórych uC można przypisać tej końcówce funkcję portu we/wy



Układ BOD (Brown-out Detector)

- Podobnie jak POR lecz ma histerezę progu wyzwalania z możliwością wyboru progu jej zadziałania

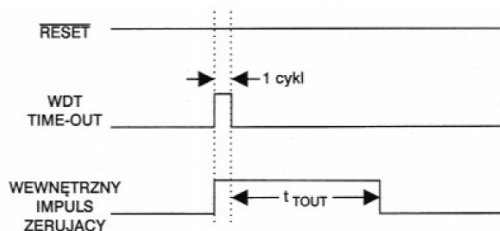


AVR – budowa

Układy zerujące/resetujące

Układ WDT (Watchdog Timer)

- Jedyne źródło impulsów zerujących sprawujące nadzór nad przebiegiem programu
- Jest to licznik impulsów z autonomicznego generatora WDT
- Przepelnienie tego licznika powoduje inicjalizację mikrokontrolera
- Umieszczenie w strategicznych punktach programu instrukcji zerujących (wdr) anuluje inicjalizację
- Mechanizm ten pozwala na nowy rozruch uC jeśli na skutek zakłóceń lub nieprzewidywalnej nieskończonej pętli „zawiesi się” – utknie w jakimś miejscu programu



AVR – budowa

Układy zerujące/resetujące

Rejestr MCUCSR

- Rejestr stanu w którym zapisywana jest informacja co było powodem ostatniego zerowania mikrokontrolera

Bit:	7	6	5	4	3	2	1	0	MCUCSR
	?	?	?	JTRF	WDRF	BORF	EXTRF	PORF	MCUSR
Zapis / Odczyt:	?	?	?	Z/O	Z/O	Z/O	Z/O	Z/O	
Wartość domyślna:	?	?	?	0/1	0/1	0/1	0/1	0/1	

- **JTRF** – komenda zerująca interfejsu JTAG (ang. *JTAG Reset Flag*),
- **WDRF** – przepełnienie licznika WDT (ang. *WatchDog Reset Flag*),
- **BORF** – nieprawidłowe napięcie zasilania (ang. *Brown-Out Reset Flag*),
- **EXTRF** – zewnętrzny sygnał $\overline{\text{RESET}}$ (ang. *External Reset Flag*),
- **PORF** – włączenie zasilania (ang. *Power-On Reset Flag*).

AVR – budowa

Zarządzanie energią

- Ważne przy zasilaniu bateryjnym
- Redukcja częstotliwości zegara taktującego oraz wykorzystanie trybów wstrzymania/uśpienia (*sleep modes*)

Wstrzymywanie systemu

- Polega na wyłączeniu CPU i zaprzestania wykonywania programu
- Nie jest kasowana zawartość rejestrów i pamięci SRAM
- Układy peryferyjne pracują w zależności od trybu uśpienia
Układ może przywrócony do działania poprzez zdarzenie przerwania (np. od peryferii) (I) albo na skutek zerowania uC (wywołanego przez któryś z modułów zerujących) (II)
- (I) - program wykonuje podprogram obsługi przerwania i przechodzi do wykonywania programu głównego, od miejsca instrukcji *sleep*
- (II) – wykonywana jest normalna procedura zerowania

AVR – budowa

Zarządzanie energią

Tryby uśpienia

- Znaczniki trybu wstrzymania:
 - SE – *sleep enable* – zezwolenie na tryby uśpienia
 - SM2...0 - *Sleep Mode select* – wybór trybu uśpienia

SM2	SM1	SM0	Nazwa trybu	Opis działania
0	0	0	<i>Idle</i>	CPU i wykonywanie programu zostają wstrzymane. Wszystkie układy peryferyjne i zegar taktujący działają jak w trybie aktywnym. Powrót do trybu aktywnego trwa cztery cykle zegarowe.
0	0	1	<i>ADC Noise Reduction</i>	Tryb wykorzystywany w trakcie pracy przetwornika analogowo-cyfrowego. Jego włączenie na czas konwersji A/C umożliwia zwiększenie dokładności wyniku poprzez wyeliminowanie wewnętrznych źródeł zakłóceń. Tryb ten, poza działaniami właściwymi dla <i>Idle</i> , powoduje także wyłączenie niektórych modułów wejścia-wyjścia (np. USART i SPI). Powrót do trybu aktywnego jest możliwy przede wszystkim na skutek przerwania wywołanego przez moduł ADC z powodu zakończenia bieżącej konwersji.
0	1	0	<i>Power-down</i>	„Najgłębszy” tryb wstrzymania. Wyłączana jest większość układów peryferyjnych oraz zegar systemowy. Działają jedynie moduły pracujące asynchronicznie względem głównego układu taktującego (np. moduł TWI). Powrót do trybu aktywnego jest wydłużony o czas potrzebny na stabilizację pracy układu zegarowego (jak dla modułu zerującego POR).
0	1	1	<i>Power-save</i>	Jak dla <i>Power-down</i> z tą różnicą, że działanie układu licznika 0 w trybie asynchronicznym jest możliwe
1	1	0	<i>Standby</i>	Tryb możliwy wyłącznie przy wykorzystaniu zewnętrznego rezonatora. Działanie takie samo jak przy trybie <i>Power-down</i> , przy czym zegar systemowy pracuje. Umożliwia to szybsze wyjście układu ze stanu uśpienia (po sześciu cyklach zegarowych).
1	1	1	<i>Extended Standby</i>	Tryb możliwy wyłącznie przy wykorzystaniu zewnętrznego rezonatora. Działanie takie samo jak przy trybie <i>Power-save</i> , przy czym zegar systemowy pracuje. Umożliwia to szybsze wyjście układu ze stanu uśpienia (po sześciu cyklach zegarowych).

AVR – budowa

Zarządzanie energią

Wstrzymywanie poszczególnych modułów

- Polega na wstrzymaniu pracy poszczególnych modułów z zachowaniem działania jednostki centralnej
- Bity sterujące rejestru PRR

7	6	5	4	3	2	1	0
PRTWI	PRTIM2	PRTIMO	PRLCD	PRTIM1	PRSPI	PRUSART0	PRADC

PRTWI- wstrzymywanie portu I2C

PRTIM2- wstrzymywanie licznika T2

PRTIMO- wstrzymywanie licznika T0

PRLCD- wyłączanie interfejsu LCD

PRTIM1- wstrzymywanie licznika T1

PRSPI- wstrzymywanie portu SPI

PRUSART0- wstrzymywanie interfejsu USART0

PRADC- wstrzymywanie przetwornika AC

AVR – Porty wejścia-wyjścia

Cechy ogólne

- Dwukierunkowość
- Trójstanowość
- Opcja wewnętrznych rezystorów podciągających
- Duża wydajność prądowa stopni wyjściowych
- Symetryczna charakterystyka wyjściowa
- Możliwość dokonywania zmian w ustawieniach poszczególnych linii na poziomie bitowym
- Kilka portów po osiem linii w każdym, które wszystkie mogą być programowane niezależnie
- Oznaczenia
- PORTA, PORTB - symboliczna nazwa portu
- PORTA1, PORTC6 - oznacza pojedynczą linię danego portu

AVR – Porty wejścia-wyjścia

Dedykowane rejestry funkcyjne

- **Rejestr DDRx** – określa kierunek danego portu

Bit:	7	6	5	4	3	2	1	0	
	DDx7	DDx6	DDx5	DDx4	DDx3	DDx2	DDx1	DDx0	DDRx
Zapis / Odczyt:	Z/O	Z/O	Z/O	Z/O	Z/O	Z/O	Z/O	Z/O	
Wartość domyślna:	0	0	0	0	0	0	0	0	

- **Wyzerowanie znacznika DDRxn (DDxn)** powoduje że dane wyprowadzenie pracuje jako **wejście**
- **Wstawienie logicznej 1 do znacznika DDRxn (DDxn)** powoduje że dane wyprowadzenie pracuje jako **wyjście**
- **bit PUD** – ustawienie go powoduje zablokowanie rezystorów podciągających na wszystkich liniach wszystkich portów

PORTxn	DDRxn	PUD	Kierunek portu	Komentarz
0	0	X	wejście	stan wysokiej impedancji (Hi-Z)
1	0	1	wejście	stan wysokiej impedancji (Hi-Z)
1	0	0	wejście	wewnętrzne podciąganie do zasilania (20...50 kΩ)
0	1	X	wyjście	wyjście w stanie L
1	1	X	wyjście	wyjście w stanie H

Możliwe konfiguracje ustawienia linii przez PORTxn, DDRxnm i PUD

AVR – Porty wejścia-wyjścia

Dedykowane rejestry funkcyjne

- Służą do sterowania pracą portów;
- Przeważnie dostępne w najniższej części przestrzeni adresowej wejścia wyjścia (0x00...0x1F) – dostęp na poziomie bitowym instrukcjami sbi i sbic

- **Rejestr PORTx** – przechowuje wartości wymuszane przez uC

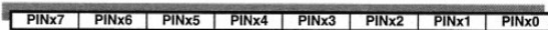
p	7	6	5	4	3	2	1	0	
	PORTx7	PORTx6	PORTx5	PORTx4	PORTx3	PORTx2	PORTx1	PORTx0	PORTx
Lub:	Px7	Px6	Px5	Px4	Px3	Px2	Px1	Px0	
Zapis / Odczyt:	Z/O	Z/O	Z/O	Z/O	Z/O	Z/O	Z/O	Z/O	
Wartość domyślna:	0	0	0	0	0	0	0	0	

- Jeśli port pracuje **w trybie wyjściowym**, znacznik rejestru PORTx określa wymuszany na nim stan logiczny
- Jeśli linia pracuje jako wejście, ustawienie na niej bitu PORTxn (Pxn) powoduje uaktywnienie wewnętrznego rezystora podciągającego tę linię do napięcia zasilania

AVR – Porty wejścia-wyjścia

Dedykowane rejestry funkcyjne

- **Rejestr PINx** – zawiera znaczniki, które odpowiadają stanowi faktycznemu portu

Bit:	7	6	5	4	3	2	1	0	
									PINx
Zapis / Odczyt:	PINx7	PINx6	PINx5	PINx4	PINx3	PINx2	PINx1	PINx0	
	O (/ Z)	O (/ Z)	O (/ Z)	O (/ Z)	O (/ Z)	O (/ Z)	O (/ Z)	O (/ Z)	
Wartość domyślna:	0	0	0	0	0	0	0	0	

- Wykorzystywanie głównie w pracy **w trybie wejściowym**
- Wysoki stan danego wejścia odpowiada wysoka wartość logiczna przyporządkowanego mu znacznika PINxn.
- Jeśli linia pracuje w jako wyjście odpowiadający jej znacznik PINxn jest kopią odpowiedniego portu PORTxn (*chyba że wyprowadzenie zostało fizycznie zwarte do przeciwnego bieguna zasilania*)
- Rejestr PINxn może być zawsze odczytany
- W niektórych uC możliwe jest zapisanie jednej z jego pozycji stanem wysokim, co spowoduje zmianę odpowiadającego mu bitu w PORTxn

AVR – Porty wejścia-wyjścia

Tryb wejściowy

- Wyzerowanie danej linii portu przez odpowiadający jej znacznik DDxn
- Ustawienie domyślne
- Na wybranych liniach działających w trybie wejściowym istnieje możliwość uaktywnienia wewnętrznego rezystora podciągającego (20-50 kΩ - rozrzut produkcyjny) do napięcia zasilania.
- Jednak przy badaniu zewnętrznego sygnału pochodzącego ze źródła o dużej oporności wewnętrznej, należy wyzerować PORTxn (zdezaktywowany rezystor podciągający) lub odpowiednio ustawić znacznik PUD
- Układ wejściowy może wprowadzać opóźnienia dochodzące do 1,5 cyklu zegara systemowego – należy odczekać przed odczytem z PINxn po ustanowieniu PORTxn, (oprócz tego opóźnienia „fizyczne”)
- Wejście z histerezą 50 mV
- Nie zaleca się pozostawianie nieużywanych linii w trybie wejściowym, bez ich podciągania (zewn. lub. wewn.) oraz bezpośredniego zwierania z masą lub zasilaniem (przy przypadkowej zmianie kierunku działania portu może doprowadzić to do uszkodzenia przez duży prąd zwarciovowy)

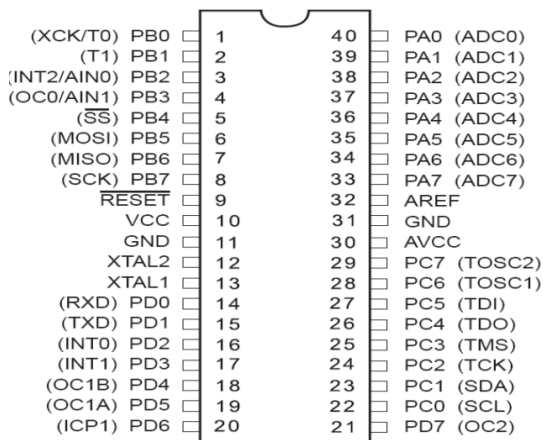
AVR – Porty wejścia-wyjścia

Tryb wyjściowy

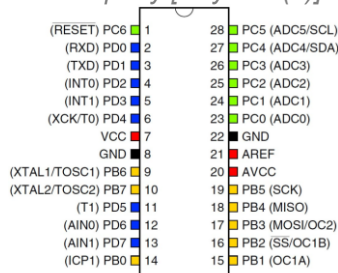
- Ustawienie danej linii portu przez odpowiadający jej znacznik DDxn
- Poziom napięcia wymuszaniu na danym wyprowadzeniu jest zależny od zawartości przyporządkowanego mu bitu PORTxn
- Jeśli dany port nie jest obciążony zbyt małą rezystancją, rejestr PINx jest kopią rejestru PORTx
- Maksymalne obciążenie jednej linii to 20 mA, wartość dopuszczalna powyżej której może nastąpić uszkodzenie to 40 mA
- Maksymalne natężenie prądu sumarycznego przepływającego przez wyprowadzenia GND i VCC mikrokontrolera zwyczajowo nie powinno przekroczyć 200 mA
- W praktyce możliwe jest bezpośrednie sterowanie diodami LED, i wyświetlaczami LED

Porty

Atmega32 posiada 4 porty A, B, C, D każdy po osiem linii z których każda może być wejściem lub wyjściem.



A np. ATmega8 posiada 3 porty [C tylko 5(6)]



Porty

Linia jako wyjście:

- W zależności od ustawienia w stan niski lub wysoki na wyjściu napięcie 0 V lub Vcc (5V)
- Wydajność prądowa 1 wyjścia 20 mA, a cały pobór prądu przez uC nie może przekroczyć 200 mA

Ustawiania portów

Trzy dedykowane rejestry sprzętowe:

- DDRx – rejestr kierunku
- PORTx – rejestr wyjściowy
- PINx – rejestr wejściowy
- *Po resecie wszystkie linie ustawione są jako wejścia*

Przykład:

DDRA = 0b00000111 - trzy najmłodsze piny są **wyjściami**, pięć najstarszych **wejsciami**
dowolne -PA0

PORTA = 0bxxxxx101 – na pinach **PA0 i PA2** pojawi się stan wysoki; na pinie PA1 stan niski

Porty

Przykład:

DDRA = 0b00000111 - trzy najmłodsze piny są **wyjściami**, pięć najstarszych **wejsciami**

Jeśli do pinów PA3-PA7 będzie podłączony klawisz podciągnięty do Vcc, to odczyt rejestru PIN da wynik:

PINA = 0b11111xxx
cokolwiek*

Wciśnięcie którekolwiek klawisza spowoduje pojawienie się 0 na pozycji danego bitu w rejestrze PINx

* W PINx odczytać także można to co zostało ustawione na wyjściach przez PORTx

UWAGA: Przy ustawieniu danego pinu jako wejście w DDRx, to ustawienie dla danego pinu wartości w PORTx, powoduje **załączenie wewnętrznego podciągnięcia do VCC** (przez rezystor 50-70k)

Porty

Ustawianie wartości w rejestrach

Niedobry przykład:

PORTA=0b00001000 – ustawia 1 na pinie 3 (bo tam chcieliśmy), ale jednocześnie 0 na wszystkich innych

Przykład dobry:

- Stosowanie operatorów bitowych
- Dla ustawienie 1 bitu na pinie 3
- `PORTA = PORTA | 0b00001000` lub krócej
`PORTA |= 0b00001000` lub jeszcze krócej
`PORTA|=(1<<3)`

```
<< - przesunięcie bitowe w lewo
>> - przesunięcie bitowe w prawo
& - bitowy iloczyn logiczny (AND)
| - bitowa suma logiczna (OR)
^ - bitowa różnica symetryczna (XOR)
~ - bitowa negacja
```

- Przykład dobry i preferowany

- Stosowanie operatorów i zdefiniowanych domyślnie wartości 1 PCx, PDx, CS01, WGM00 które są nazwami bitów w rejestrach

`PORTA|=(1<<PA3)`

```
#define PD0 0
#define PD1 1
#define PD2 2
#define PD3 3
#define PD4 4
#define PD5 5
#define PD6 6
#define PD7 8
```

Porty

Ustawianie wartości w rejestrach

Na takiej zasadzie (warto by samemu przeanalizować skąd to się bierze) będziemy ustawiać 1, ustawiać 0 i zmieniać stan konkretnych bajtów

Aby ustawić bit (1)

`BAJT |= (1<< nr_bitu);`

Aby wyzerować bit (0)

`BAJT &= ~(1<<nr_bitu);`

Aby zmienić stan bitu (0->1 lub 1->0)

`BAJT ^= (1<<nr_bitu);`

Aby ustawić 2 bity (i analogicznie więcej)

`BAJT |= (1<< nr_bitu) | (1<< nr_bitu);`

Aby wyzerować 2 (i analogicznie więcej)

`BAJT &= ~( (1<<nr_bitu) | (1<<nr_bitu) );`

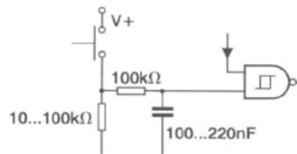
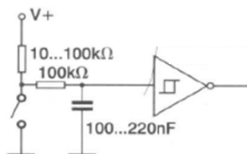
Porty

Linia jako wejście:

- Wejście jest w stanie wysokiej impedancji (HiZ)

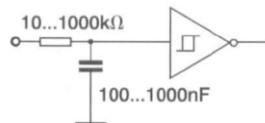
Co można podłączyć na wejście:

- Wyjście innego układu cyfrowego o stanach kompatybilnych z CMOS: 0V – V_{cc} (5V)
- Przełącznik (klawisz) mechaniczny – mogą pojawić się oscylacje sygnału błędnie interpretowane przez program, stąd podobny układ wejściowy:



(ale istnieją rozwiązania programowe)

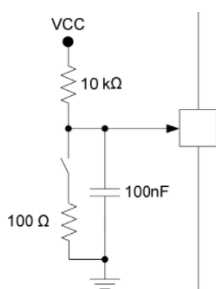
- Wolnozmienny sygnał analogowy, ale wtedy najlepiej bramką buforową z przerzutnikiem Schmitta i filtrem



Porty

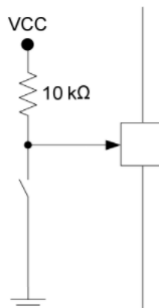
Linia jako wejście:

Wzorcowe
podłączenie
przełącznika do
wejścia

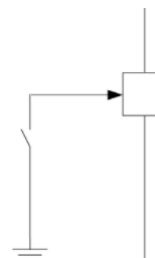


Układy wejścia z podciągnięciem do V_{cc}; na wejściu nie pojawi się stan nieustalony przy braku wciśnięcia przycisku

Dobre podłączenie
przełącznika do
wejścia



Złe podłączenie
przełącznika do
wejścia



Uwaga:
Złe jeśli
programowo nie
podciągniemy
wejście do V_{cc}

Porty

Obsługa klawiszy micro-switch

Bezpośrednio po wciśnięciu i zwolnieniu przycisku pojawiają się drgania styku mogące powodować fałszywe sygnały dla mikroprocesora



Najprostszym sposobem programowym zabezpieczającym przed tym zjawiskiem jest wprowadzenie powtórnego, opóźnionego sprawdzenia stanu przycisku:

```
// klawisz K1
#define KEY_PIN (1<<PC6)
#define KEY_DOWN !(PINC & KEY_PIN)

uint8_t klawisz_wcisniety(void); // deklaracja funkcji
// ***** pętla główna main()
...

// definicja funkcji
uint8_t klawisz_wcisniety(void)
{
    if( KEY_DOWN ) // klawisz wciśnięty ?
    {
        delay_ms(80); // czas drgań styków
        if( KEY_DOWN ) return 1; // jeśli wciśnięty? zakończ funkcję - rezultat = 1
    }
    return 0; // jeśli nie wciśnięty klawisz, zakończ funkcję, rezultat = 0
}
```

AVR – Porty wejścia-wyjścia

Alternatywne funkcje wyprowadzeń

(XCK/T0) PB0	1	40	PA0 (ADC0)	-	Prawie każde wyprowadzenie portów we/wy w ATmega ma przyporządkowaną co najmniej jedną funkcję alternatywną, może zostać przyporządkowana jednemu z wewnętrznych układów peryferyjnych – wyjścia multipleksowane
(T1) PB1	2	39	PA1 (ADC1)		
(INT2/AIN0) PB2	3	38	PA2 (ADC2)		
(OC0/AIN1) PB3	4	37	PA3 (ADC3)		
(SS) PB4	5	36	PA4 (ADC4)		
(MOSI) PB5	6	35	PA5 (ADC5)		
(MISO) PB6	7	34	PA6 (ADC6)		
(SCK) PB7	8	33	PA7 (ADC7)		
RESET	9	32	AREF		
VCC	10	31	GND		
GND	11	30	AVCC	-	Funkcje alternatywne są uaktywniane przeważnie automatycznie przy włączeniu danego modułu wewnętrznego, ustawienia portów są ignorowane
XTAL2	12	29	PC7 (TOSC2)		
XTAL1	13	28	PC6 (TOSC1)		
(RXD) PD0	14	27	PC5 (TDI)		
(TXD) PD1	15	26	PC4 (TDO)		
(INT0) PD2	16	25	PC3 (TMS)		
(INT1) PD3	17	24	PC2 (TCK)		
(OC1B) PD4	18	23	PC1 (SDA)		
(OC1A) PD5	19	22	PC0 (SCL)		
(ICP1) PD6	20	21	PD7 (OC2)		

Schemat wyprowadzeń ATmega32